An Integer Square-Root Instruction for the CVE2 RISC-V Core

Author's Personal Copy Technical Report

AHMED EZ-ZOUINE, M.Sc.

September 2026

Project website: ahmedez-zouine.github.io/cve2-custom-isa-extension/

Abstract

This report describes an unsigned integer square-root instruction added to the CVE2 two-stage RV32IMC core. The ALU contains sixteen restoring steps connected as combinational logic. The instruction uses the normal decoder and register writeback path. It passes the supplied tests, extra boundary tests and instruction integration checks. On the same simulated workload, the average drops from 247 software cycles to 23 cycles per root with ISQRT, a ratio of about 10.7. Routed LUT use on an Artix-7 rises by 40.8%. The extended design fails the 20 ns setup target. The results show that the instruction works in the tested RTL and explain its FPGA cost; FPGA timing closure is still incomplete.

Keywords: RISC-V, CVE2, custom instruction, integer square root, FPGA.

I. Instruction and processor integration

For unsigned input x, ISQRT returns $r = \lfloor \sqrt{x} \rfloor$, with $0 \leq$ x $\leq 2^{32}$ - 1 and $0 \leq$ r $\leq$ 65,535. The defining condition is $\text{r}^2 \leq \text{x} < (\text{r}+1)^2$. The result is zero-extended into the 32-bit destination register. The instruction uses the RISC-V custom-0 major opcode, reserved for non-standard extensions [1]. Table I lists the fields.

TABLE I
ISQRT INSTRUCTION ENCODING

31:25	24:20	19:15	14:12	11:7	6:0
0000000	00000	rs1	000	rd	0001011

The extension changes three RTL files: `cve2_pkg.sv`, `cve2_decoder.sv` and `cve2_alu.sv`. The package defines custom-0 and ALU_ISQRT. The decoder checks that funct7, rs2 and funct3 are zero, reads rs1, and selects the square-root result for rd. Other custom-0 encodings raise an illegal-instruction exception. The supplied testbench and core configuration stay the same, and CV-X-IF is unused. Figure 1 shows the ISQRT data path. The complete core organization is shown next in Figure 2 [2].

The core reads the old value of rs1 before it writes rd at the clock edge. This also works when rd and rs1 name the same register. The existing register file discards writes to x0. In this configuration, the writeback block passes the result through without adding a pipeline stage. ISQRT adds no dedicated execution stall. Section III checks this behavior in simulation.

II. Restoring square root and implementation

A square-root circuit can build its answer two input bits at a time. Earlier work includes formally verified non-restoring designs [3]. This design uses the restoring method. After k pairs, P_k is the part of the input already read, Q_k is its root, and R_k is the remainder. The root and remainder start at zero. They follow this relation:

$$P_k = Q_k^2 + R_k, \qquad 0 \leq R_k \leq 2Q_k. \tag{1}$$

When the next pair b_k arrives, the prefix becomes $4P_k + b_k$. The next root bit is either zero or one. The update is:

$$T = 4R_k + b_k, \qquad D = 4Q_k + 1,$$
$$(Q_{k+1}, R_{k+1}) = \begin{cases} (2Q_k + 1, T - D), & T \geq D, \\ (2Q_k, T), & T < D. \end{cases} \tag{2}$$

If T < D, the new remainder is at most $4Q_k = 2Q_{k+1}$. Otherwise, T $\leq 8Q_k + 3$ implies T - D $\leq 4Q_k + 2 = 2Q_{k+1}$. Both cases keep the relationship in (1) and its remainder bound. After sixteen pairs, $P_{16} =$ x, so the bound implies $Q_{16}^2 \leq$ x $< (Q_{16} + 1)^2$. This proves the arithmetic rule. It does not prove the behavior of the whole processor.

The SystemVerilog `always_comb` block sets all temporary values before running a loop with sixteen iterations. Synthesis turns this loop into a chain of logic, as shown in Figure 3. Before the last step, $Q \leq$ 32,767 and $R \leq$ 65,534. Therefore $T \leq$ 262,139 and $D \leq$ 131,069, which fit in the unsigned 32-bit temporary values. The root needs only sixteen bits. The 32-bit result has zeros in its upper bits.

Using the existing ALU path keeps the control changes small. The cost is a long chain: each comparison and subtraction depends on the previous step. As Section IV shows, this path takes too long for the chosen 20 ns FPGA clock period.

III. Verification and cycle measurements

The final review rebuilt revision 95090a4 from a clean source export with Verilator 5.054, RISC-V GCC 16.2.0 and GNU Make 3.81. It did not reuse the simulator or program binaries from the working checkout. Every test in Table II passed. The saved logs include hashes of the source and result files [6], [7]. Later documentation edits leave these tested sources unchanged.

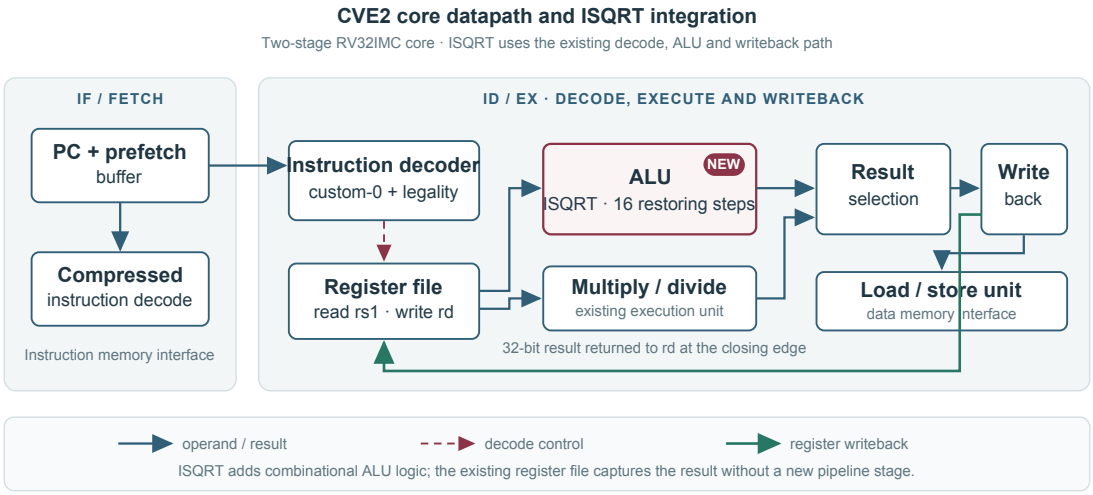


Fig. 1. ISQRT integration through the existing decoder, ALU and writeback path. The sixteen arithmetic steps are combinational. The decoder selects ISQRT, the ALU calculates the root, and the existing writeback path stores it. No new pipeline stage is added.

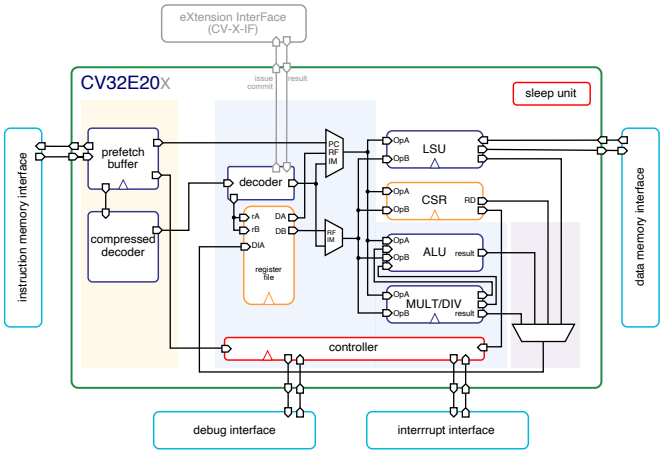


Fig. 2. CVE2 core organization and its instruction, data and control interfaces.
This view shows the full core. ISQRT fits inside the ALU; the memory and control interfaces keep their existing roles.

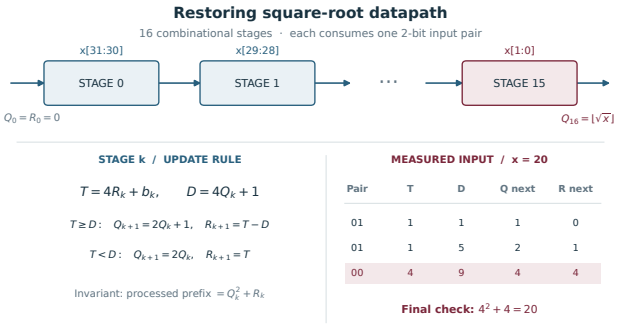


Fig. 3. Restoring square-root steps for input 20. The table shows the arithmetic at each step.
Each step reads two input bits and adds one root bit. For input 20, the final root is 4 because $4^2 \leq 20 < 5^2$.

The last three input pairs are `01`, `01` and `00`; the leading zero pairs leave the root and remainder at zero. These three steps produce $(Q, R) = (1, 0)$, then $(2, 1)$, then $(4, 4)$. The final remainder checks the result: $20 = 4^2 + 4$. Only the root is written to the destination register. All sixteen steps form one combinational chain, so the intermediate values shown here are arithmetic steps, not separate clock cycles.

TABLE II
FUNCTIONAL VERIFICATION RESULTS

Program	Checks	Coverage
sw_isqrt	5	Supplied software reference cases
hw_isqrt	2,007	Known/random inputs, in-place write, dependent roots
isqrt_corners	43	Boundaries, x0, invalid encodings, trap recovery
isqrt_integration	200,705	Square boundaries, full-width inputs, dependencies

The integration test checks the values just below, equal to and just above each of the 65,535 positive square boundaries. It also checks zero and UINT32_MAX. Another 4,096 inputs cover the full input width. These checks use the square-root inequalities and 64-bit products, so squaring 65,536 does not overflow. A separate instruction sequence tests load, ISQRT and store, followed by dependent multiply, unsigned divide and compressed addition. The supplied C reference uses the same algorithm as the RTL. Agreement with that reference alone would not rule out a shared algorithm error.

Figure 4 shows one execution of ISQRT with input 20. The trace checks the result and the register write at the clock edge.

Figure 5 compares the benchmark cycle counts. Both benchmark programs use the same 256 inputs after `rand_seed(7)`. Both produce checksum 10,996,072. The printed averages, rounded to whole cycles, are 247 cycles per root for software and 23 for ISQRT. Their ratio is about 10.7. These counts include the loop, input generation and checksum work. They describe the complete benchmark, not the latency of one ISQRT instruction. They also do not establish a 10.7× gain in FPGA execution time.

The tests cover selected RV32IMC instruction interactions and four invalid encodings. They do not cover every custom encoding or register pair. A full ISA regression, sustained throughput testing and operation on a physical

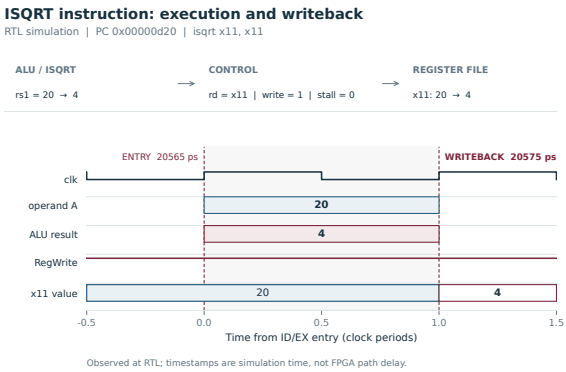


Fig. 4. RTL trace of isqrt x11, x11 at PC 0x00000d20. The trace checks valid execution, register write, stall = 0 and illegal = 0.

The stored value of x11 changes from 20 to 4 at the next rising clock edge. This trace shows RTL behavior; it does not measure FPGA signal delay.

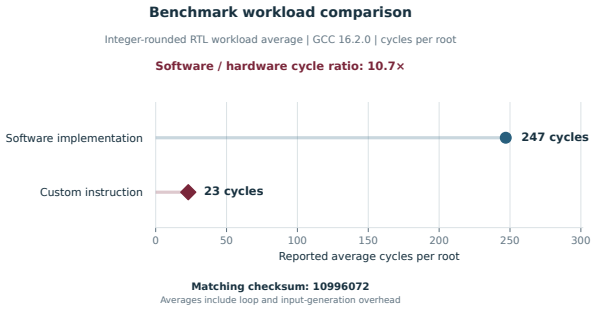


Fig. 5. Reported average cycles per root from the matched 256-input workload.

Both programs process the same inputs. The shorter line shows fewer cycles for the complete workload, including its loop and checksum.

board remain outside this evidence.

IV. FPGA area, timing and power

Vivado 2024.2 builds both designs for the Artix-7 xc7a35tcsg324-1 [4]. The core is built out of context (OOC), meaning that the surrounding system is not fully implemented. Both builds use RV32IMC, the fast multiplier, a flip-flop register file and zero optional HPM counters. The wrapper exposes the instruction and data buses and synchronizes reset release. The FPGA source list defines SYNTHESIS, leaves out tracing and testbench modules, and replaces the simulation clock gate with a 7-series BUFGCE.

The baseline uses the three original RTL files from import af1e7b4. The wrapper and build inputs match the extended design. Both builds use a 20 ns clock target and I/O delays of 0 ns minimum and 4 ns maximum. These delays assume a surrounding synchronous system. The flow runs `opt_design`, `place_design`, `phys_opt_design` and `route_design`, using Default directives and four threads [4], [6].

Table III includes the core and wrapper. Timing comes from the routed designs. Power comes from the corrected run with assumed switching activity. Each power value is rounded separately, so the displayed components may not add up to the displayed total.

TABLE III
WHOLE-DESIGN FPGA RESOURCES, TIMING AND ESTIMATED POWER

Metric	Original core	With ISQRT
Synthesis LUTs	3,034	4,178
Routed LUTs	2,987	4,205
Routed flip-flops	1,961	1,965
Routed DSP / RAMB36 / RAMB18	1 / 0 / 0	1 / 0 / 0
Overall setup WNS (ns)	-0.884	-29.556
Filtered register setup slack (ns)	+0.058	-29.556
Hold WHS (ns)	-2.232	-2.168
Total on-chip estimate (mW)	86	101
Dynamic estimate (mW)	17	32
Device-static estimate (mW)	68	68

A. Area and timing findings

Figure 6 compares routed resource use. The extension adds 1,218 LUTs, a 40.8% increase. Both builds use one DSP and no block RAM. Routing finishes for all 4,806 baseline nets and all 5,883 extended nets that need routing, with zero routing errors.

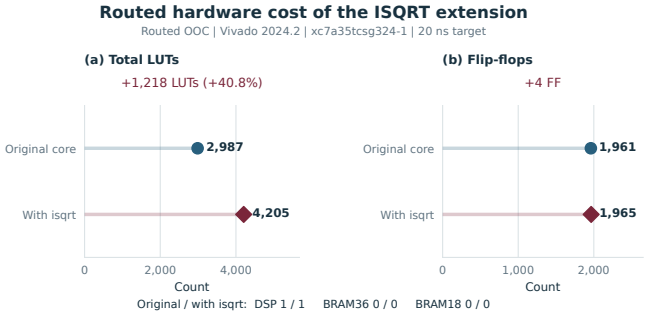


Fig. 6. Routed FPGA resource use for the original core and the ISQRT extension.

LUTs hold the logic; flip-flops hold state. The extension mainly increases logic use. Both designs still use one DSP and no block RAM.

Figure 7 shows the timing checks. Both designs fail overall setup and hold. The baseline's separate register setup check has +0.058 ns slack, even though its overall setup check fails.

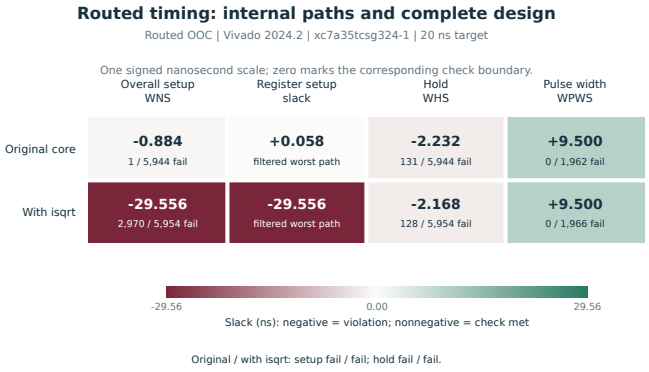


Fig. 7. Routed setup, hold and pulse-width slack at the 20 ns target.

Negative slack means a timing check fails. Both designs fail overall setup and hold. The baseline passes the separate register setup check even though its overall setup check fails.

Figure 8 shows how LUT use and setup slack change between synthesis and routing. These are two saved stages of the same flow for each variant.

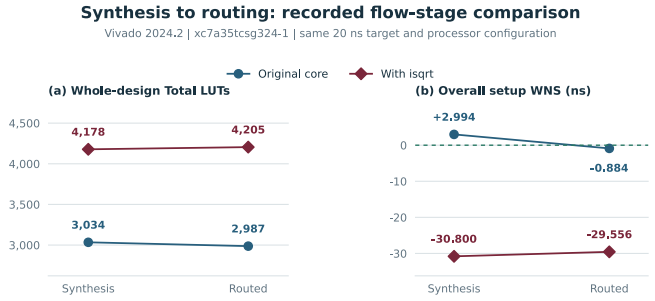


Fig. 8. LUT use and worst setup slack after synthesis and routing. Each line joins two saved build stages.

The two points show synthesis and routing results. Routing changes the numbers, but the extended design still has a large setup violation.

Figure 9 separates logic delay from wire delay on the slowest reported register paths. The extended path passes through ISQRT and takes 49.387 ns: 15.399 ns in logic and 33.988 ns in wires, across 83 logic levels. This includes selection logic and other parts of the path. It is not the delay of the square-root block alone. The baseline path takes 19.672 ns. The paths have different start and end registers.

B. Power conditions and evidence limits

Figure 10 compares the power estimates. The corrected power run assumes typical process, commercial grade, a fixed junction temperature of 25 °C, 1.000 V V_{ccint} and 1.800 V V_{ccaux}. At an assumed 50 MHz, 68 data and handshake input bits use 6.25 million transitions per second and a 0.5 probability of being high. Two control bits stay high; 23 interrupt, debug and error bits stay low. Activity was checked on all 93 non-clock inputs before and after propagation. Vivado reports Medium confidence for both estimates. Total estimated power rises by 15 mW, about 17.4%. These values are not measured board power or workload energy [5], [6].

The external clock source and interface routes are not fully defined. Vivado reports HD.CLK_SRC and boundary-route warnings. The baseline Constraints 18-8777 warning is also unresolved, and the successful extended implementation launch log was not recovered. The saved reports and checkpoint audits support the numbers shown here, but these gaps remain. The results do not establish a maximum working FPGA clock frequency.

V. Iterative architecture and trade-offs

An iterative design could reuse one restoring step over several cycles. Registers would hold the remaining input, partial root, remainder and iteration count. Each cycle would read two more bits, form $4Q + 1$, choose the next root bit and save the result. This is a proposed design. Only the combinational version has been implemented and measured. Table IV compares it with the proposal.

TABLE IV
IMPLEMENTED AND ALTERNATIVE ARCHITECTURES

Aspect	Unrolled unit	Iterative proposal
Arithmetic	Sixteen dependent restoring steps	One reused decision stage
State	No iteration state	Input, root, remainder, counter and control
Instruction execution	One observed RTL execution interval	Sixteen arithmetic iterations; capture/completion may add cycles
Area	Measured +1,218 routed LUTs for the extended core	Less replicated arithmetic expected; extra state and control
Timing	Internal path fails 20 ns target	Shorter arithmetic path expected; timing must be measured
Control	Existing ALU result and writeback selection	Start, busy, completion, stalled execution and deferred writeback

A. How a multicycle unit would fit CVE2

The decoder would start each valid operation once and save its input and destination register. ID/EX control would hold the instruction while the unit is busy and prevent an early register write. Later instructions would wait so they cannot read an unfinished result. Once ready, the root would be written exactly once and normal execution would resume. The existing multiply/divide completion path offers a starting point, but the shared resources and result selection still need review.

When rd equals rs1, the saved input must remain available until all iterations finish. When rd is x0, the operation can finish but its register write must be discarded. Invalid custom-0 encodings must trap before the unit starts. Reset, flush, debug and interrupt handling must follow the core's completion rules. In particular, a cancelled operation must not write an old result later. These changes belong inside the execution control; the assignment excludes CV-X-IF.

Sixteen arithmetic steps do not necessarily mean sixteen instruction cycles. Saving the input could share the first step, and writeback could share the last. Separate start or finish states would take extra cycles. New tests would need to check consecutive operations, dependent instructions, reset and cancellation, as well as the arithmetic.

B. Expected benefit and remaining measurement

Reusing one step should reduce duplicated arithmetic and shorten the logic within each cycle. It also adds registers and control, and another core path may become the slowest one. Meeting 20 ns would still need a new implementation run. A higher clock rate might offset the extra cycles, but only a benchmark at a supported clock rate can show the total execution time. No area, frequency or speedup result has been measured for this proposal.

A smaller arithmetic unit does not automatically use less energy per result. It may switch less logic in each cycle, but it runs for more cycles and must clock its

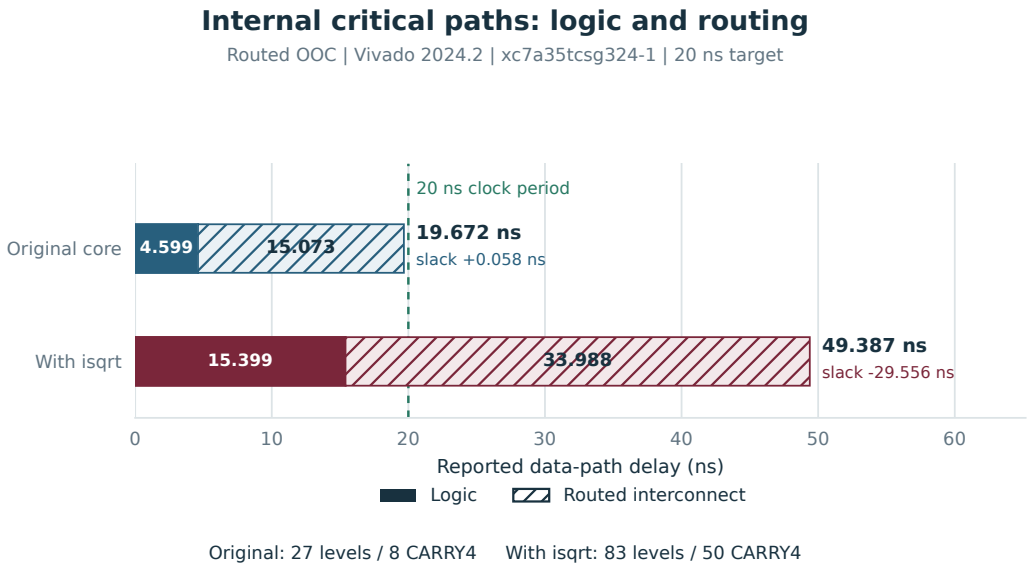


Fig. 9. Logic and wire delay on the slowest reported register paths. The two paths have different endpoints; the extended path passes through ISQRT.

The solid part is logic delay; the hatched part is wire delay. The extended path takes 49.387 ns, well beyond the 20 ns clock period.

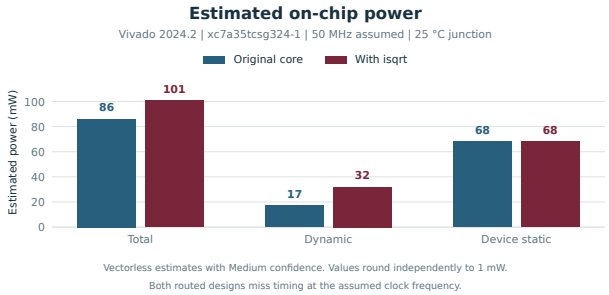


Fig. 10. Estimated on-chip power using assumed switching activity.

Both estimates use the same operating assumptions. Dynamic power rises with the added logic. The displayed device-static estimate stays at 68 mW.

new registers. A fair comparison needs realistic switching activity, working clock rates and total workload time for both designs. The current timing result gives a clear reason to investigate this option.

VI. Conclusion

The ISQRT instruction works through CVE2's existing decoder, ALU and register writeback path. It passes the reported tests and reduces the benchmark average from 247 to 23 cycles per root. The FPGA cost is 40.8% more routed LUTs and a long path that fails the 20 ns target. Power is still an estimate based on assumed activity. The next useful comparison is an iterative unit, measured for area, timing, workload cycles and energy.

VII. Reproduction and artifact availability

The repository contains the RTL, test logs, scripts, figure data and FPGA reports [6], [7]. Sections III and IV give the tool versions and build settings. The README files explain the simulation, FPGA flow and archive review. Reproducing the baseline needs the full Git history to retrieve the original core sources. Routed checkpoints remain on the Vivado host. These results describe the saved revisions; a new physical implementation may produce different results.

A. AI-use acknowledgment

OpenAI ChatGPT/Codex helped with RTL and test edits, debugging, scripts, documentation, figures and report drafts. The results reported here come from simulation logs and Vivado output. AI assistance does not provide an independent correctness check.

References

[1] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, version 20240411, Sec. 35.3.2, Apr. 2024. [Online]. Available: https://docs.riscv.org/reference/isa/v20240411/

[2] OpenHW Foundation, *CORE-V CVE2 User Manual*, "Pipeline Details" and "Instruction Decode and Execute," draft documentation. [Online]. Available: https://docs.openhwgroup.org/projects/cve2-user-manual/en/latest/03_reference/pipeline_details.html

[3] J. O'Leary, M. Leeser, J. Hickey, and M. Aagaard, "Non-restoring integer square root: A case study in design by principled optimization," in *Theorem Provers in Circuit Design: Theory, Practice and Experience*, ser. Lecture Notes in Computer Science, vol. 901. Springer, 1995, pp. 52–71. doi: https://doi.org/10.1007/3-540-59047-1_42.

[4] AMD, *Vivado Design Suite User Guide: Design Analysis and Closure Techniques*, UG906, 2024.2 ed., Dec. 2024. [Online]. Available: https://docs.amd.com/r/2024.2-English/ug906-vivado-design-analysis

[5] AMD, *Vivado Design Suite User Guide: Power Analysis and Optimization*, UG907, 2024.2 ed., Nov. 2024. [Online]. Available: https://docs.amd.com/r/2024.2-English/ug907-vivado-power-analysis-optimization

[6] A. Ez-Zouine, *CVE2 Custom ISA Extension: Source Code and FPGA Evidence*, revision b24909e, 2026. [Online]. Available: https://github.com/ahmedez-zouine/cve2-custom-isa-extension

[7] A. Ez-Zouine, "ISQRT functional verification and benchmark artifacts," project repository, revisions 95090a4 and b24909e, 2026. [Online]. Available: https://github.com/ahmedez-zouine/cve2-custom-isa-extension/tree/Master/output